

JAVA · UNIDAD 8 · CHULETARIO

Clases Abstractas · Modificador Final · Interfaces

1. CLASES ABSTRACTAS

Características

- No se puede instanciar directamente
- Puede tener métodos abstractos y concretos
- Puede tener atributos de instancia
- Obliga a subclases a implementar lo abstracto

Sintaxis

```
abstract class Animal {
    protected String nombre;
    public Animal(String nombre) {
        this.nombre = nombre;
    }
    // sin cuerpo → obliga a subclase
    public abstract void hacerSonido();
    // método concreto compartido
    public void dormir() {
        System.out.println("durmiendo");
    }
}
```

Subclase — implementación obligatoria

```
class Perro extends Animal {
    public Perro(String n) { super(n); }
    @Override
    public void hacerSonido() {
        System.out.println("Guau");
    }
}
```

Uso en Main

```
// Animal a = new Animal(); ← ERROR
Animal a = new Perro("Firulais");
a.hacerSonido(); // Guau
a.dormir();      // Firulais durmiendo
```

- La clase abstracta fuerza el diseño correcto de la jerarquía

Cuándo usarla

- Varias clases comparten código base común
- Quieres forzar implementación en hijas

2. MODIFICADOR FINAL

Método final — no se sobrescribe

```
class Persona {
    public final void mostrar() {
        System.out.println("Fijo");
    }
}
class Est extends Persona {
    // public void mostrar(){} ← ERROR
}
```

Clase final — no se hereda

```
final class Utilidades { }
// class X extends Utilidades {} ← ERROR
```

Atributo final — valor constante

```
class Config {
    final int PUERTO = 8080;
    void f() {
        // PUERTO = 9090; ← ERROR
    }
}
```

- Seguridad y valores inamovibles
- Clases utilitarias (Math, String...)

3. INTERFACES

Características

- Contrato 100 % abstracto (sin estado)
- Solo constantes, no atributos de instancia
- Una clase puede implementar varias a la vez

Definición e implementación

```
interface Reprodutor {
    void reproducir();
    void parar();
}
class RepAudio implements Reprodutor {
    @Override
    public void reproducir() {
        System.out.println("Play");
    }
    @Override
    public void parar() {
        System.out.println("Stop");
    }
}
```

4. ABSTRACTA VS INTERFAZ

Característica	Abstracta	Interfaz
Métodos con código	Sí	No (usual)
Atributos instancia	Sí	No
Constructor	Sí	No
Herencia múltiple	No	Sí
Uso	Base común	Contrato

Interfaz múltiple

```
interface Nadador { void nadar(); }
interface Volador { void volar(); }
```

```
class Pato
    implements Nadador, Volador {
    public void nadar() {
        System.out.println("Splash");
    }
    public void volar() {
        System.out.println("Fly");
    }
}
```

Abstracta + Interfaz combinadas

```
abstract class Vehiculo {
    public abstract void arrancar();
}
interface Electrico { void cargar(); }
```

```
class CocheE extends Vehiculo
    implements Electrico {
    public void arrancar() {
        System.out.println("Silencio");
    }
    public void cargar() {
        System.out.println("Cargando");
    }
}
```

- extends 1 clase + implements N interfaces (a la vez)

JAVA · UNIDAD 8 · CHULETARIO

Genéricos (Generics) · Resumen global · Ejercicios

5. GENÉRICOS — PROBLEMA Y SOLUCIÓN

¿Por qué genéricos?

- Reutilizar código para distintos tipos
- Evitar casting inseguro
- Detectar errores de tipo en compilación

Sin genéricos — peligroso

```
class Caja {
    private Object obj;
    public void set(Object o) { obj=o; }
    public Object get() { return obj; }
}

Caja c = new Caja();
c.set("Hola");
// ERROR solo visible en ejecución:
Integer n = (Integer) c.get(); // ← BOOM
```

Con genéricos — seguro

```
class Caja<T> {
    private T obj;
    public void set(T o) { obj = o; }
    public T get() { return obj; }
}
```

Uso de Caja<T>

```
Caja<String> cs = new Caja<>();
cs.set("Hola");
String s = cs.get(); // sin cast
```

```
Caja<Integer> ci = new Caja<>();
ci.set(42);
int n = ci.get();    // sin cast
// ci.set("texto"); ← ERROR compilación
```

✓ Error detectado en compilación, no en ejecución

5. GENÉRICOS — AVANZADO

Par<K, V> — dos parámetros de tipo

```
class Par<K, V> {
    private K clave;
    private V valor;
    public Par(K c, V v) {
        clave = c; valor = v;
    }
    public K getClave() { return clave; }
    public V getValor() { return valor; }
}

// Uso:
Par<String,Integer> p = new Par<>("Edad",30);
System.out.println(p.getClave()); // Edad
System.out.println(p.getValor()); // 30
```

Letras convencionales

Letra	Uso
T	Tipo genérico (Type)
E	Elemento en listas
K	Clave (Key)
V	Valor (Value)
N	Número

Colecciones Java — usan genéricos

```
// Lista tipada
ArrayList<String> lista = new ArrayList<>();
lista.add("Java");
String s = lista.get(0); // sin cast
```

```
// Mapa clave→valor
HashMap<String, Integer> mapa = new HashMap<>();
mapa.put("edad", 25);
int edad = mapa.get("edad");
```

Object vs Genérico

Sin genérico	Con genérico
Usa Object	Usa <T>
Casting necesario	Sin casting
Error en ejecución	Error al compilar
Menos legible	Más claro

6. RESUMEN & EJERCICIOS

Ideas clave globales

Palabra	Significado
abstract	Debe implementarse en subclase
final	No se puede modificar ni heredar
interface	Contrato que debe cumplirse
<T>	Funciona con cualquier tipo seguro

Ejercicios propuestos

#	Descripción
1	Clase abstracta Animal → Perro
2	Figura abstracta → Rectangulo, Circulo
3	CuentaBancaria con método final
4	Clase Utilidades declarada final
5	Configuracion con atributo final
6	Interfaz Volador + clase Ave
7	Pato implementa Nadador + Volador
8	Vehiculo abstracto + interfaz Electrico
9	Caja<T> con String y Integer
10	Par<K,V> con distintos tipos

Checklist de comprensión

- Crear clase abstracta con método abstracto
- No instanciar clase abstracta directamente
- Usar final en clase, método y atributo
- Definir e implementar una interfaz
- Distinguir abstracta vs interfaz
- Crear clase genérica Caja<T>
- Entender errores de tipo en compilación
- Usar ArrayList<T> y HashMap<K,V>